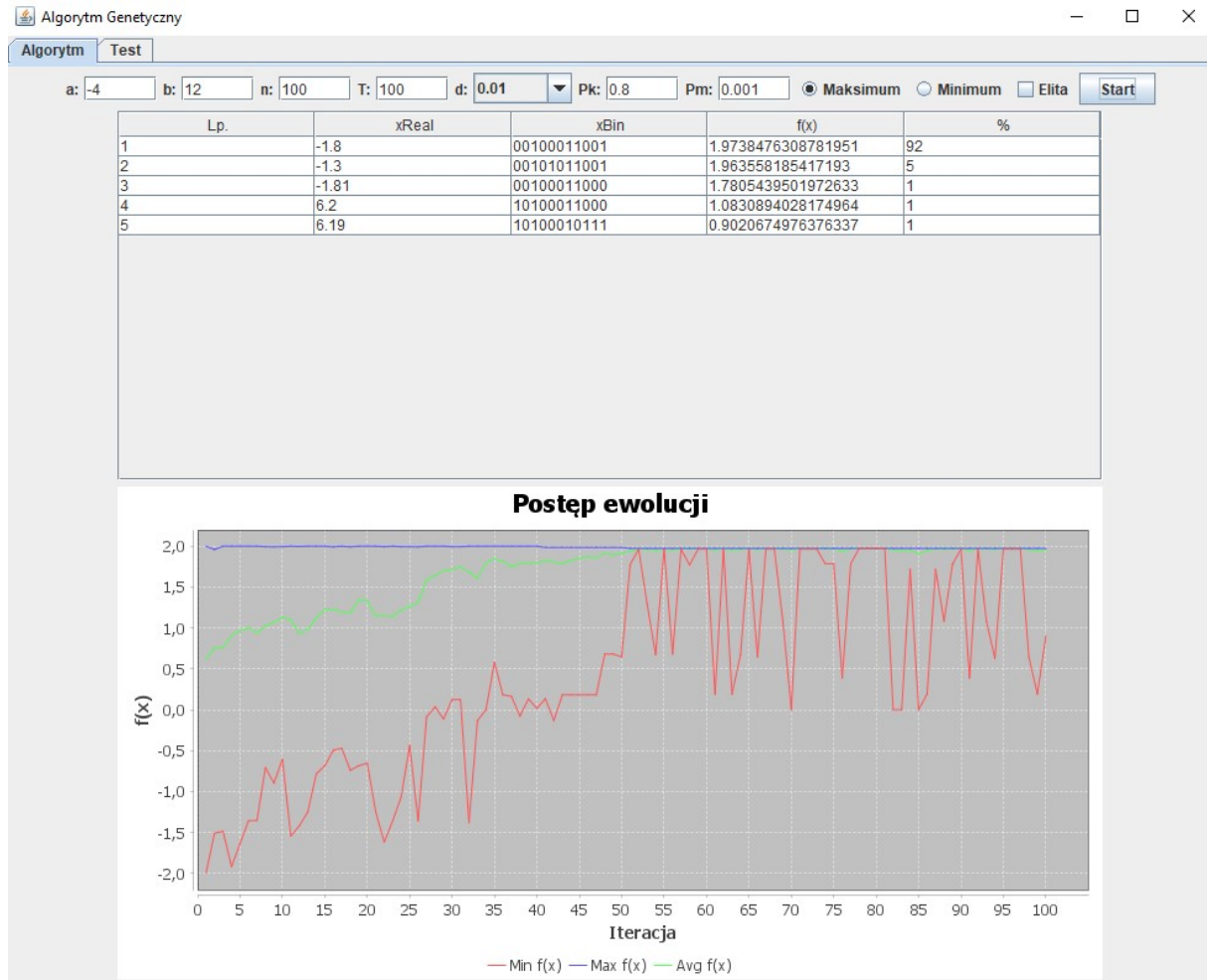


DOKUMENTACJA PROJEKTU

Algorytm genetyczny - optymalizacja funkcji

Desktopowa aplikacja prezentująca działanie algorytmu genetycznego



Java • Swing • JFreeChart • Maven

Bartosz Ochnio | 2026

1. Informacje o projekcie

Projekt jest desktopową aplikacją prezentującą działanie algorytmu genetycznego na przykładzie optymalizacji funkcji jednej zmiennej. Program umożliwia wyszukiwanie maksimum albo minimum funkcji w przedziale określonym przez użytkownika.

Użytkownik może ustawić liczebność populacji, liczbę iteracji, dokładność obliczeń oraz prawdopodobieństwo krzyżowania i mutacji. Wyniki są prezentowane w tabeli oraz na wykresie pokazującym przebieg ewolucji.

Cel projektu: implementacja i wizualizacja etapów algorytmu genetycznego wykorzystywanego do optymalizacji funkcji matematycznej.

Najważniejsze funkcjonalności

- wyszukiwanie minimum albo maksimum funkcji;
- binarne kodowanie chromosomów;
- selekcja metodą koła ruletki;
- krzyżowanie jednopunktowe i mutacja bitowa;
- opcjonalne zachowywanie najlepszego osobnika;
- prezentacja końcowej populacji i statystyk ewolucji;
- automatyczne porównywanie zestawów parametrów.

Technologie

Technologia	Zastosowanie
Java	logika programu i algorytmu
Swing	graficzny interfejs użytkownika
JFreeChart	wykres przebiegu ewolucji
Maven	zarządzanie projektem i zależnościami
IntelliJ IDEA	tworzenie i uruchamianie aplikacji

2. Problem optymalizacyjny

Zadaniem algorytmu jest znalezienie argumentu x z zadanego przedziału, dla którego funkcja osiąga możliwie największą albo najmniejszą wartość. W projekcie wykorzystywana jest funkcja:

$$f(x) = \cos(20\pi x) - \sin(x)$$

Ze względu na losowe generowanie populacji, selekcję i mutację wyniki kolejnych uruchomień mogą się nieznacznie różnić.

Parametry algorytmu

Parametr	Znaczenie
a	dolna granica przedziału wartości x
b	górną granicę przedziału wartości x
n	liczebność populacji
T	liczba iteracji algorytmu
d	dokładność reprezentacji wartości
Pk	prawdopodobieństwo wyboru do krzyżowania
Pm	prawdopodobieństwo mutacji pojedynczego bitu
Maksimum / Minimum	kierunek optymalizacji
Elita	zachowanie najlepszego znalezionej osobnika
Interpretacja: większa populacja i liczba iteracji zwiększają liczbę wykonywanych obliczeń, natomiast parametry Pk i Pm wpływają na różnorodność kolejnych pokoleń.	

3. Przebieg algorytmu

1. **Inicjalizacja** — tworzona jest populacja n losowych wartości z przedziału $[a, b]$.
2. **Kodowanie** — wartość rzeczywista jest zamieniana na wartość całkowitą, a następnie na binarny chromosom.
3. **Ocena** — dla każdego osobnika obliczana jest wartość funkcji $f(x)$.
4. **Selekcja** — osobniki są losowane metodą koła ruletki; lepiej przystosowane otrzymują większą szansę wyboru.
5. **Krzyżowanie** — wybrani rodzice są łączeni parami za pomocą krzyżowania jednopunktowego.
6. **Mutacja** — każdy bit może zostać odwrócony z prawdopodobieństwem P_m .
7. **Elitaryzm** — po włączeniu opcji Elita najlepszy dotychczasowy osobnik jest zachowywany.
8. **Kolejne pokolenie** — operacje są powtarzane T razy, po czym zwracana jest populacja końcowa.

Reprezentacja chromosomu

Długość ciągu binarnego zależy od przedziału i dokładności:

$$l = \lceil \log_2((b - a) / d + 1) \rceil$$

4. Struktura projektu

Projekt podzielono na interfejs użytkownika, modele danych oraz serwis zawierający logikę obliczeniową. Pozwala to oddzielić prezentację wyników od wykonywania algorytmu.

Klasa	Odpowiedzialność
Form	interfejs, odczyt parametrów, tabela wyników i wykres
GeneticMath	inicjalizacja, ocena, selekcja, krzyżowanie, mutacja, elitaryzm i testy
Chromosome	dane pojedynczego osobnika i wyniki operatorów genetycznych
Xreal / XInt / Xbin	rzeczywista, całkowita i binarna reprezentacja chromosomu
EvolutionStats	minimalna, maksymalna i średnia wartość $f(x)$ w iteracjach
GaResult	końcowa populacja wraz ze statystykami
Test	parametry i średni wynik testowanej konfiguracji
Main	punkt startowy aplikacji

Podział pakietów

- Gui - formularz aplikacji;
- Model - obiekty opisujące chromosomy, wyniki i statystyki;
- Service - logika algorytmu genetycznego;
- org.example - klasa uruchamiająca program.

Główna sekwencja iteracji: selekcja → krzyżowanie i mutacja → opcjonalne przywrócenie elity → zapis statystyk.

5. Obsługa aplikacji

W zakładce „Algorytm” użytkownik wprowadza parametry, wybiera kierunek optymalizacji i decyduje o zastosowaniu elitaryzmu. Przycisk „Start” uruchamia obliczenia oraz odświeża tabelę i wykres

The screenshot shows a software interface for a genetic algorithm. At the top, there are input fields for parameters: 'a: -4', 'b: 12', 'n: 100', 'T: 100', 'd: 0.01' (with a dropdown arrow), 'Pk: 0.8', and 'Pm: 0.001'. To the right of these fields are three radio buttons: 'Maksimum' (selected), 'Minimum', and 'Elita'. A 'Start' button is located at the far right. Below the parameters is a table with 5 columns: 'Lp.', 'xReal', 'xBin', 'f(x)', and '%'. The table contains 5 rows of data representing the final population.

Lp.	xReal	xBin	f(x)	%
1	-1.8	00100011001	1.9738476308781951	92
2	-1.3	00101011001	1.963558185417193	5
3	-1.81	00100011000	1.7805439501972633	1
4	6.2	10100011000	1.0830894028174964	1
5	6.19	10100010111	0.9020674976376337	1

Rysunek 1. Parametry i tabela populacji końcowej.

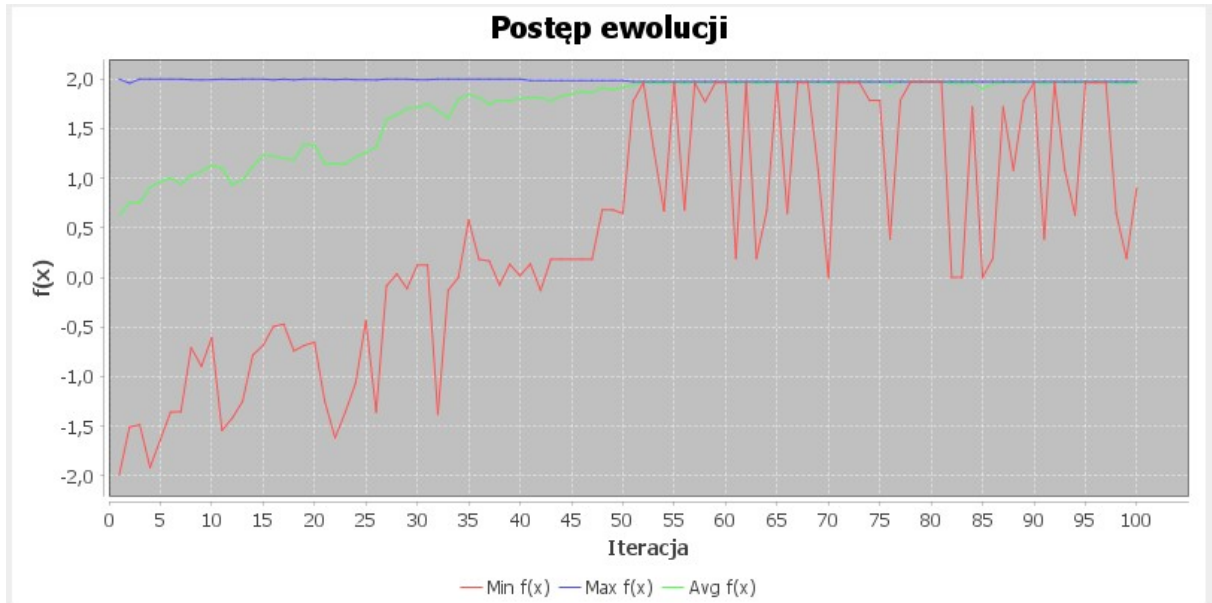
Tabela wyników

Kolumna	Znaczenie
xReal	wartość rzeczywista reprezentowana przez osobnika
xBin	binarna reprezentacja chromosomu
f(x)	wartość optymalizowanej funkcji
%	udział osobników o danej wartości w populacji

Osobniki o takiej samej wartości funkcji są grupowane, dlatego tabela może zawierać mniej wierszy niż liczebność populacji.

6. Prezentacja postępu ewolucji

Wykres przedstawia minimalną, maksymalną i średnią wartość funkcji w każdym pokoleniu. Pozwala obserwować, czy populacja zbliża się do korzystniejszego rozwiązania i jak szybko przebiega ten proces.



Rysunek 2. Wykres minimalnej, maksymalnej i średniej wartości funkcji.

- Min $f(x)$ - najniższa wartość funkcji w populacji;
- Max $f(x)$ - najwyższa wartość funkcji w populacji;
- Avg $f(x)$ - średnia wartość funkcji w populacji.

Moduł testowy

Moduł testowy porównuje 16 konfiguracji algorytmu utworzonych z różnych wartości liczebności populacji **N**, prawdopodobieństwa krzyżowania **P_k**, prawdopodobieństwa mutacji **P_m** oraz liczby iteracji **T**. Każda konfiguracja jest uruchamiana dziesięciokrotnie, a kolumna Favg(x) przedstawia średnią wartość funkcji w populacji końcowej. Wyniki są uporządkowane od najwyższej do najniższej wartości.

Algorytm Genetyczny

Algorytm Test

☐ Elita

Lp.	N	P _k	P _m	T	Favg(x)
1	60	0,80	0,0001	100	1.983050513191607
2	60	0,60	0,0001	50	1.9821747381755803
3	60	0,60	0,0010	100	1.9795755103699015
4	60	0,60	0,0001	100	1.9716324761719144
5	30	0,80	0,0001	100	1.9617755430518038
6	30	0,60	0,0001	50	1.9584957152635514
7	30	0,60	0,0010	100	1.957423331230802
8	60	0,80	0,0010	100	1.9486364675143613
9	30	0,60	0,0001	100	1.9484335000504214
10	30	0,80	0,0001	50	1.9468159186938874
11	60	0,80	0,0001	50	1.9466402379886667
12	30	0,60	0,0010	50	1.9453261963647115
13	60	0,60	0,0010	50	1.9389535452568425
14	30	0,80	0,0010	50	1.9317079185090544
15	30	0,80	0,0010	100	1.922363905601203
16	60	0,80	0,0010	50	1.8821203597106195

Rysunek 3. Wyniki automatycznego porównania parametrów algorytmu.

7. Uruchomienie projektu

Aplikację można uruchomić jako gotowy pakiet dla systemu Windows albo bezpośrednio z kodu źródłowego w środowisku programistycznym.

Uruchomienie gotowej aplikacji w systemie Windows

Gotowa wersja programu zawiera środowisko uruchomieniowe oraz plik [app.bat](#), dlatego nie wymaga instalowania środowiska Java, Mavena ani IntelliJ IDEA.

Aby uruchomić aplikację:

1. Pobierz archiwum zawierające gotową wersję programu.
2. Rozpakuj całą zawartość archiwum do wybranego katalogu.
3. Zachowaj oryginalną strukturę podkatalogów i plików danych.
4. Uruchom plik: [app.bat](#)

Nie należy przenosić samego pliku [app.bat](#) poza katalog aplikacji. Program korzysta z dołączonego środowiska uruchomieniowego oraz plików danych znajdujących się w pozostałych podkatalogach.

Uruchomienie z kodu źródłowego

Wymagania

- zainstalowane środowisko Java;
- Maven;
- IntelliJ IDEA lub inne środowisko obsługujące projekty Maven.

Instrukcja

1. **Otwórz projekt** - zaimportuj folder zawierający plik pom.xml do środowiska programistycznego.
2. **Pobierz zależności** - poczekaj na zakończenie synchronizacji projektu Maven.
3. **Uruchom aplikację** - wykonaj metodę main z klasy Main.
4. **Wprowadź dane** - ustaw parametry i użyj przycisku Start.

8. Podsumowanie

Projekt przedstawia praktyczne wykorzystanie algorytmu genetycznego do optymalizacji funkcji matematycznej. Zaimplementowano binarne kodowanie chromosomów, selekcję metodą koła ruletki, krzyżowanie jednopunktowe, mutację bitową oraz opcjonalny elitaryzm.

Interfejs pozwala konfigurować algorytm, analizować populację końcową i śledzić zmiany wartości funkcji w kolejnych pokoleniach. Moduł testowy umożliwia dodatkowo porównywanie wielu konfiguracji parametrów.

Możliwe kierunki rozwoju

- wprowadzanie własnej funkcji celu;
- dodanie innych metod selekcji i krzyżowania;
- eksport wyników do pliku CSV;